BEZPIECZEŃSTWO APLIKACJI INTERNETOWYCH

LABORATORIUM / PROJEKT 2026

OPIS PROJEKTÓW DLA STUDENTÓW

Co wykonać • co oddać • co zaprezentować • z jakich standardów skorzystać

dr inż. Jarosław Homa, MBA, CISO

Akademia WSB • semestr zimowy 2026

NAJWAŻNIEJSZA ZASADA

Projekt ma być praktyczny: zespół identyfikuje problem bezpieczeństwa, pokazuje dowód techniczny, wdraża zabezpieczenie i wykonuje test ponowny potwierdzający skuteczność poprawki.

Projekt realizowany jest zespołowo, w grupach 2–3 osobowych. Każdy zespół wybiera jeden z tematów P1–P12 albo proponuje własny temat, który wymaga akceptacji prowadzącego.

ZESPÓŁ I ŚRODOWISKO	OBOWIĄZKOWE ELEMENTY
• 2–3 osoby w zespole • wyłącznie środowisko własne, laboratoryjne, celowo podatne lub jednoznacznie autoryzowane • zakaz testowania publicznych serwisów bez zgody właściciela • środowisko musi być odtwarzalne na podstawie instrukcji i repozytorium	• dowód techniczny problemu lub działania kontroli • realna remediacja: poprawka w kodzie, konfiguracji, architekturze lub procesie • test ponowny po zmianie: BEFORE → AFTER • mapowanie do minimum dwóch standardów/metodyk • raport 8–12 stron + repozytorium / artefakty + prezentacja z demo

Warunki akceptacji projektu

1. Legalny zakres	testy wykonywane tylko w dozwolonym środowisku
2. Odtwarzalność	możliwość uruchomienia projektu i powtórzenia scenariusza
3. Dowód techniczny	request, log, ekran, test, fragment kodu lub inny materiał potwierdzający wynik
4. Remediacja	wdrożona poprawka lub kompletna zmiana konfiguracji/architektury
5. Test ponowny	ponowne wykonanie kluczowego scenariusza po poprawce
6. Standardy i ryzyko	minimum dwa odniesienia + ocena ryzyka i priorytetu
7. Raport i materiały	raport 8–12 stron oraz repozytorium/załączniki
8. Obrona	10–12 minut + demonstracja; każdy członek zespołu rozumie cały projekt

Harmonogram

04.10.2026	18.10.2026	25.10.2026	21.11.2026
START zespół + temat + zakres	CHECKPOINT środowisko + model + dowody	**PREFEROWANE ZALICZENIE** **demo + raport + repo + retest**	TERMIN KOŃCOWY poprawki / pozostałe projekty

Karta projektu: wypełniana osobno. Rekomendowany mikrotermin wersji 0.1: 11.10.2026.

Poniższe tematy są propozycjami. Zakres konkretnego zespołu może zostać zawężony po uzgodnieniu z prowadzącym. Nazwy angielskie pozostawiono w nawiasach, ponieważ występują w dokumentacji technicznej i narzędziach.

ID	Temat	Główne zagadnienie
P1	**Klucze dostępu 2026 (Passkeys)**	uwierzytelnianie i odzyskiwanie dostępu
P2	**Autoryzacja w aplikacji wielodostępnej (Multi-tenant)**	autoryzacja i separacja danych
P3	**Ochrona interfejsu API (API Shield)**	bezpieczeństwo REST / GraphQL API
P4	**SBOM i kontrola łańcucha dostaw**	zależności, CVE i łańcuch dostaw
P5	**Potok bezpieczeństwa DevSecOps**	automatyzacja kontroli w CI/CD
P6	**Bezpieczne przesyłanie plików (Secure Upload)**	upload/download i walidacja plików
P7	**Utwardzanie przeglądarki 2026 (Browser Hardening)**	XSS, CSRF, CSP i nagłówki HTTP
P8	**SSRF i granica zaufania chmury**	SSRF i kontrola ruchu wychodzącego
P9	**Monitoring i obserwowalność bezpieczeństwa**	logi, korelacja i alerty
P10	**Bezpieczne zachowanie przy awarii (Fail Secure) / A10:2025**	bezpieczne zachowanie podczas błędu
P11	**Bezpieczeństwo komunikacji WebSocket**	WebSocket i autoryzacja wiadomości
P12	**Bezpieczne programowanie wspomagane AI**	bezpieczeństwo kodu generowanego przez AI

P1. Klucze dostępu 2026 (Passkeys)

WebAuthn i klucze dostępu (passkeys) w porównaniu z jednorazowymi kodami TOTP

CEL PROJEKTU

Porównać nowoczesne logowanie kluczami kryptograficznymi z klasycznym uwierzytelnianiem kodem TOTP i sprawdzić, gdzie powstają słabe ścieżki odzyskiwania dostępu.

CO MA POKAZAĆ DEMONSTRACJA

Rejestracja poświadczenia, logowanie, próba nieautoryzowanego użycia oraz scenariusz odzyskiwania dostępu — przed i po wdrożeniu zabezpieczeń.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• lokalna aplikacja z kontami użytkowników i sesją • WebAuthn/passkeys jako główna metoda logowania • TOTP jako wariant porównawczy albo metoda zapasowa • scenariusze: rejestracja, logowanie, utrata urządzenia, odzyskanie dostępu, unieważnienie poświadczenia • ocena sesji, ciasteczek, blokad i komunikatów błędów	• diagram przepływu rejestracji i logowania • minimum 6 scenariuszy testowych • dowód działania poprawnego oraz odrzuconych scenariuszy nieautoryzowanych • opis mechanizmu odzyskiwania dostępu i ryzyk fallbacku • test ponowny po poprawkach

Wymagane odniesienia do standardów

OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dotyczące uwierzytelniania, sesji i zarządzania poświadczeniami
OWASP Top 10 A07:2025	kategoria najważniejszych ryzyk dotycząca uwierzytelniania — w projekcie: kategoria błędów uwierzytelniania
W3C WebAuthn Level 3	standard uwierzytelniania kryptograficznego w przeglądarce — w projekcie: standard techniczny używany do tworzenia i używania kluczy kryptograficznych

Wskazówka: Nie sprowadzajcie projektu do „działa logowanie”. Kluczowe jest porównanie ryzyka oraz bezpieczna obsługa odzyskiwania dostępu i sesji.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P2. Autoryzacja w aplikacji wielodostępnej (Multi-tenant)

BOLA/IDOR, kontrola ról RBAC/ABAC i separacja danych organizacji

CEL PROJEKTU

Zbudować małą aplikację obsługującą wielu klientów i udowodnić, że użytkownik jednego klienta nie może zobaczyć ani zmienić danych innego klienta.

CO MA POKAZAĆ DEMONSTRACJA

Próba odczytu lub modyfikacji cudzego zasobu przez zmianę identyfikatora, a następnie ten sam test po poprawieniu kontroli dostępu.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• minimum dwie organizacje/tenanty i co najmniej dwie role • zasoby posiadające właściciela lub tenant_id • macierz uprawnień: kto może czytać, tworzyć, zmieniać, usuwać • testy poziomej i pionowej eskalacji uprawnień • porównanie RBAC i ewentualnie ABAC dla jednego scenariusza	• macierz uprawnień • minimum 8 testów negatywnych • dowód BOLA/IDOR w wersji podatnej • poprawiona autoryzacja po stronie serwera • test ponowny każdego krytycznego scenariusza

Wymagane odniesienia do standardów

OWASP Top 10 A01:2025	kategoria ryzyk dotycząca kontroli dostępu — w projekcie: nieprawidłowa kontrola dostępu
OWASP API Security Top 10:2023	lista najważniejszych ryzyk specyficznych dla API — w projekcie: BOLA jest jednym z głównych ryzyk API
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania weryfikujące autoryzację i separację dostępu

Wskazówka: Nie wystarczy ukryć przycisków w interfejsie. Autoryzacja musi być egzekwowana na backendzie dla każdego zasobu.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P3. Ochrona interfejsu API (API Shield)

REST/GraphQL, walidacja danych, autoryzacja i ograniczanie liczby żądań

CEL PROJEKTU

Zaprojektować małe API i zabezpieczyć je przed typowymi problemami: nieuprawnionym dostępem, nadmiernym ujawnianiem danych, błędną walidacją i nadużyciem liczby żądań.

CO MA POKAZAĆ DEMONSTRACJA

Kilka wywołań API pokazujących problem w wersji podatnej, następnie poprawione odpowiedzi po włączeniu autoryzacji, walidacji i ograniczeń.

CO NALEŻY WYKONAĆ

- 4–6 endpointów REST lub niewielkie API GraphQL
- autoryzacja zasobów i operacji
- walidacja schematu danych wejściowych
- ograniczenie liczby żądań dla wybranych operacji
- kontrola nadmiarowego ujawniania danych i masowego przypisywania pól
- bezpieczne komunikaty błędów

CO NALEŻY ODDAĆ

- dokumentacja endpointów i model danych
- scenariusze pozytywne i negatywne
- dowody minimum 3 klas problemów API
- poprawki w kodzie lub konfiguracji
- test ponowny po poprawkach

Wymagane odniesienia do standardów

OWASP API Security Top 10:2023	lista najważniejszych ryzyk specyficznych dla API — w projekcie: główne źródło kategorii ryzyka specyficznych dla API
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dotyczące walidacji, kontroli dostępu i usług webowych
OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: szerszy kontekst ryzyk aplikacyjnych

Wskazówka: Nie opierajcie oceny API wyłącznie narzędziem automatycznym. Najciekawsze problemy wynikają z logiki biznesowej i autoryzacji.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P4. SBOM i kontrola łańcucha dostaw

wykaz składników oprogramowania (SBOM), analiza zależności (SCA) i podatności CVE

CEL PROJEKTU

Pokazać, że aplikacja może mieć bezpieczny własny kod, a nadal być narażona przez biblioteki i zależności.

CO MA POKAZAĆ DEMONSTRACJA

Wygenerowanie SBOM, wykrycie podatnej zależności oraz działanie reguły, która ostrzega lub blokuje budowanie przy nieakceptowalnym ryzyku.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• aplikacja z kontrolowanym zestawem zależności • wygenerowanie SBOM w formacie CycloneDX lub równoważnym • analiza zależności narzędziem SCA • identyfikacja CVE i ocena faktycznej ekspozycji • reguła blokująca/ostrzegająca dla wysokiego ryzyka • udokumentowany update/replace/mitigate dla co najmniej jednej zależności	• SBOM przed i po zmianach • raport zależności i CVE • uzasadnienie priorytetu naprawy • automatyczna bramka bezpieczeństwa lub skrypt kontrolny • test ponowny po aktualizacji zależności

Wymagane odniesienia do standardów

OWASP Top 10 A03:2025	kategoria ryzyk łańcucha dostaw oprogramowania — w projekcie: ryzyka łańcucha dostaw oprogramowania
NIST SSDF v1.1	ramy bezpiecznego procesu wytwarzania oprogramowania — w projekcie: praktyki ochrony procesu i artefaktów wytwórczych
CWE/CVSS	klasyfikacja typów błędów oraz ocena istotności podatności — w projekcie: klasyfikacja i techniczna ocena podatności

Wskazówka: Nie wystarczy wkleić wynik skanera. Student ma wyjaśnić wpływ konkretnej zależności i decyzję naprawczą.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P5. Potok bezpieczeństwa DevSecOps

analiza kodu (SAST), zależności (SCA), testy dynamiczne (DAST), sekrety i CI/CD

CEL PROJEKTU

Włączyć automatyczne kontrole bezpieczeństwa do procesu budowania aplikacji, tak aby błędy były wykrywane możliwie wcześnie i powtarzalnie.

CO MA POKAZAĆ DEMONSTRACJA

Commit lub build uruchamia zestaw testów; problem bezpieczeństwa powoduje ostrzeżenie albo zatrzymanie procesu, a po poprawce potok przechodzi pomyślnie.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• repozytorium z niewielką aplikacją • SAST – analiza kodu • SCA – analiza zależności • skanowanie sekretów • DAST na uruchomionej aplikacji testowej • prosta polityka bramki jakości, np. blokada dla wysokiego ryzyka	• plik konfiguracji CI/CD • wynik każdej klasy testu • przykład błędu powodującego niepowodzenie potoku • poprawka i zielony przebieg po zmianie • krótkie uzasadnienie, które testy uruchamiać na każdym commitcie, a które okresowo

Wymagane odniesienia do standardów

NIST SSDF v1.1	ramy bezpiecznego procesu wytwarzania oprogramowania — w projekcie: ramy organizujące bezpieczny proces wytwórczy
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: źródło wymagań, które można częściowo automatyzować
OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: kontekst klas problemów wykrywanych w procesie

Wskazówka: Unikajcie projektu typu „uruchomiliśmy narzędzie i jest raport”. Oceniana jest integracja procesu, polityka i reakcja na wynik.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P6. Bezpieczne przesyłanie plików (Secure Upload)

typ MIME, rozszerzenia, kwarantanna i ochrona przed manipulacją ścieżką

CEL PROJEKTU

Zaprojektować mechanizm przesyłania i pobierania plików, który nie ufa nazwie ani deklarowanemu typowi pliku i nie pozwala wydostać się poza kontrolowany obszar danych.

CO MA POKAZAĆ DEMONSTRACJA

Wersja podatna przyjmuje niebezpieczny lub mylący plik albo błędną ścieżkę; poprawiona wersja odrzuca, izoluje lub bezpiecznie przetwarza przypadek.

CO NALEŻY WYKONAĆ

- upload i download w lokalnej aplikacji
- weryfikacja rozszerzenia i typu MIME
- generowanie nazw po stronie serwera
- zapis poza katalogiem publicznym aplikacji
- ograniczenie rozmiaru i liczby plików
- ochrona przed path traversal przy pobieraniu
- opcjonalna kwarantanna lub skanowanie antywirusowe

CO NALEŻY ODDAĆ

- macierz dozwolonych typów
- minimum 6 przypadków negatywnych
- dowód problemu w wersji podatnej
- bezpieczna konfiguracja i kod po poprawce
- test ponowny

Wymagane odniesienia do standardów

OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dotyczące wejścia, plików i kontroli dostępu
OWASP WSTG 4.2	przewodnik i scenariusze testowania bezpieczeństwa aplikacji webowych — w projekcie: scenariusze testowania przesyłania i obsługi danych
OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: kategorie konfiguracji i wstrzyknięć

Wskazówka: Nie musicie demonstracji niebezpiecznego kodu poza laboratorium. Wystarcza bezpieczny plik testowy i dowód mechanizmu ochrony.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P7. Utwardzanie przeglądarki 2026 (Browser Hardening)

DOM XSS, polityka bezpieczeństwa treści (CSP), CSRF i nagłówki HTTP

CEL PROJEKTU

Pokazać podatność warstwy przeglądarkowej i zbudować zestaw zabezpieczeń ograniczających wykonanie niepożądanego kodu i nadużycie sesji.

CO MA POKAZAĆ DEMONSTRACJA

Kontrolowany przykład XSS lub CSRF w aplikacji laboratoryjnej, następnie ten sam scenariusz z poprawionym kodem, CSP i innymi nagłówkami.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• lokalna aplikacja z prostym frontendem • co najmniej jeden przypadek XSS • mechanizm ochrony przed CSRF dla operacji zmieniającej stan • CSP i wybrane nagłówki bezpieczeństwa • ustawienia cookies: Secure, HttpOnly, SameSite • porównanie zachowania przed i po zmianie	• zrzut żądania/odpowiedzi i dowód podatności • konfiguracja nagłówków • fragment poprawionego kodu • test ponowny w przeglądarce i narzędziu proxy • krótkie wyjaśnienie, która warstwa zabezpiecza jaki scenariusz

Wymagane odniesienia do standardów

OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: kontekst wstrzyknięć i błędnej konfiguracji
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dla przeglądarki, sesji i danych wyjściowych
OWASP WSTG 4.2	przewodnik i scenariusze testowania bezpieczeństwa aplikacji webowych — w projekcie: praktyczne scenariusze testowe

Wskazówka: Nie ograniczajcie się do serwisu typu securityheaders. Student ma pokazać mechanizm w aplikacji i wpływ konkretnej zmiany.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P8. SSRF i granica zaufania chmury

fałszowanie żądań po stronie serwera (SSRF), lista dozwolonych celów i ruch wychodzący

CEL PROJEKTU

Zasymulować funkcję pobierającą adres URL po stronie serwera i pokazać, jak może ona przekroczyć granicę zaufania do usług wewnętrznych.

CO MA POKAZAĆ DEMONSTRACJA

Podatna aplikacja wykonuje niepożądane połączenie do lokalnej usługi wewnętrznej; poprawiona wersja blokuje niedozwolony cel i kontroluje przekierowania.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• lokalna aplikacja pobierająca URL po stronie backendu • osobna symulowana usługa wewnętrzna / metadata service • lista dozwolonych domen lub adresów • kontrola przekierowań • kontrola rozwiązywania DNS i prywatnych zakresów adresów • opcjonalna reguła ruchu wychodzącego (egress)	• diagram granic zaufania • dowód połączenia do zasobu, który powinien być niedostępny • poprawiona walidacja i konfiguracja sieciowa • testy przekierowań i adresów prywatnych • test ponowny

Wymagane odniesienia do standardów

OWASP WSTG 4.2	przewodnik i scenariusze testowania bezpieczeństwa aplikacji webowych — w projekcie: metodyka testowania żądań po stronie serwera
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania walidacji URL i komunikacji wychodzącej
OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: kontekst projektowania i walidacji danych

Wskazówka: Całość wykonujcie lokalnie. Nie używać realnych usług metadanych chmurowych ani zewnętrznych celów bez zgody.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P9. Monitoring i obserwowalność bezpieczeństwa

logi bezpieczeństwa, korelacja zdarzeń, alerty i panel monitoringu

CEL PROJEKTU

Zaprojektować takie logowanie zdarzeń, aby po incydencie dało się odpowiedzieć: kto, kiedy, co próbował zrobić i czy system to zauważył.

CO MA POKAZAĆ DEMONSTRACJA

Seria kontrolowanych zdarzeń — błędne logowania, zmiana roli, odrzucone żądanie — pojawia się w logach i wyzwala prosty alert lub dashboard.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• zdefiniowany schemat zdarzenia bezpieczeństwa • logowanie uwierzytelniania, odmów dostępu i zmian uprawnień • identyfikator korelacyjny żądania/sesji • ochrona danych wrażliwych w logach • minimum 2 reguły alarmowe • prosty panel monitoringu lub czytelny raport zdarzeń	• katalog zdarzeń i pól logu • przykładowe logi poprawnych i niepoprawnych operacji • reguły wykrywania • dashboard lub raport korelacyjny • test ponowny po zmianie sposobu logowania

Wymagane odniesienia do standardów

OWASP Top 10 A09:2025	kategoria ryzyka dotycząca logowania i alarmowania — w projekcie: ryzyka związane z brakiem logowania i alarmowania
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dotyczące bezpiecznego rejestrowania i wykrywania zdarzeń

Wskazówka: Nie skupiajcie się samego efektownego dashboardu. Najważniejsze są właściwe zdarzenia, pola, korelacja i brak wycieku sekretów.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P10. Bezpieczne zachowanie przy awarii (Fail Secure) / A10:2025

wyjątki, limity czasu i zasobów oraz kontrolowana degradacja

CEL PROJEKTU

Sprawdzić, czy aplikacja w sytuacji błędu, przeciążenia lub niedostępności zależności pozostaje w bezpiecznym stanie i nie otwiera dostępu.

CO MA POKAZAĆ DEMONSTRACJA

Wywołanie kontrolowanego błędu lub timeoutu w wersji podatnej prowadzi do niebezpiecznego zachowania lub wycieku; poprawiona wersja odmawia bezpiecznie i zapisuje zdarzenie.

CO NALEŻY WYKONAĆ

- minimum 3 scenariusze awarii: timeout, wyjątek, brak zależności lub limit zasobu
- kontrolowana obsługa błędów i kodów HTTP
- brak ujawniania stack trace/sekretów
- domyślna odmowa przy problemie z autoryzacją
- limity czasu i zasobów dla wybranej funkcji
- logowanie i alertowanie istotnych awarii

CO NALEŻY ODDAĆ

- tabela scenariuszy awarii i oczekiwanego zachowania
- dowód niebezpiecznego zachowania w wersji początkowej
- poprawiona obsługa wyjątków/timeoutów
- testy automatyczne lub skrypt powtarzający scenariusze
- test ponowny

Wymagane odniesienia do standardów

OWASP Top 10 A10:2025	kategoria ryzyka dotycząca nieprawidłowej obsługi sytuacji wyjątkowych — w projekcie: nieprawidłowa obsługa sytuacji wyjątkowych
OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dotyczące błędów, odporności i kontroli bezpieczeństwa

Wskazówka: Nie mylcie z testami wydajnościowymi. Chodzi o bezpieczeństwo zachowania podczas błędu, nie o maksymalną liczbę użytkowników.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P11. Bezpieczeństwo komunikacji WebSocket

Origin, uwierzytelnianie, autoryzacja wiadomości i ochrona przed powtórzeniem (replay)

CEL PROJEKTU

Zabezpieczyć długotrwałe połączenie WebSocket, w którym sama poprawna sesja nie wystarcza do bezpiecznej obsługi wszystkich wiadomości.

CO MA POKAZAĆ DEMONSTRACJA

Mini-czat lub strumień zdarzeń: nieautoryzowana wiadomość lub połączenie jest przyjmowane w wersji podatnej, a po poprawce zostaje odrzucone.

CO NALEŻY WYKONAĆ	CO NALEŻY ODDAĆ
• prosta aplikacja WebSocket z 2–3 typami wiadomości • uwierzytelnienie przy zestawianiu połączenia • autoryzacja dla poszczególnych operacji/kanałów • walidacja Origin • limit liczby/rozmiaru wiadomości • próba ponownego odtworzenia komunikatu lub operacji	• diagram połączenia i przepływu wiadomości • minimum 6 scenariuszy negatywnych • dowód błędu autoryzacji lub replay w wersji podatnej • poprawione reguły serwera • test ponowny

Wymagane odniesienia do standardów

OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: wymagania dla komunikacji, uwierzytelniania i autoryzacji
OWASP WSTG 4.2	przewodnik i scenariusze testowania bezpieczeństwa aplikacji webowych — w projekcie: metody testowania bezpieczeństwa aplikacji i kanałów webowych

Wskazówka: Nie wystarczy zabezpieczyć sam handshake. Każda wiadomość biznesowa powinna mieć kontrolowany kontekst użytkownika i uprawnień.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

P12. Bezpieczne programowanie wspomagane AI

kod generowany przez AI, weryfikacja według ASVS/WSTG i porównanie przed/po

CEL PROJEKTU

Sprawdzić, czy kod wygenerowany przez model AI spełnia konkretne wymagania bezpieczeństwa i jak zmienia się po świadomej weryfikacji oraz poprawkach.

CO MA POKAZAĆ DEMONSTRACJA

Model AI generuje niewielką funkcję webową; zespół znajduje konkretne problemy, poprawia kod i pokazuje testy bezpieczeństwa przed i po zmianie.

CO NALEŻY WYKONAĆ

- jedna funkcja webowa o ograniczonym zakresie, np. logowanie, upload lub endpoint API
- jawnie zapisany prompt i wynik modelu
- mapowanie wymagań do ASVS
- testy zgodne z WSTG lub API Top 10
- przegląd kodu oraz automatyczne testy pomocnicze
- poprawiona wersja i ponowna weryfikacja

CO NALEŻY ODDAĆ

- prompt i niezmieniona wersja pierwotna kodu
- lista problemów bezpieczeństwa z dowodami
- mapowanie do standardów
- poprawiony kod
- porównanie wyników testów przed/po oraz wnioski o ograniczeniach użycia AI

Wymagane odniesienia do standardów

OWASP ASVS 5.0.0	standard wymagań bezpieczeństwa aplikacji — w projekcie: obiektywny katalog wymagań dla oceny wygenerowanego kodu
OWASP WSTG 4.2	przewodnik i scenariusze testowania bezpieczeństwa aplikacji webowych — w projekcie: procedury sprawdzenia bezpieczeństwa działającej funkcji
OWASP Top 10:2025	lista najważniejszych klas ryzyka aplikacji webowych — w projekcie: klasyfikacja wykrytych problemów

Wskazówka: Projekt nie może być wyłącznie porównaniem odpowiedzi modeli. Musi obejmować działający kod, testy bezpieczeństwa, poprawkę i test ponowny.

Przed oddaniem sprawdźcie: czy potraficie w 2–3 zdaniach wyjaśnić problem, pokazać dowód BEFORE, opisać poprawkę i pokazać wynik AFTER.

Pojęcie	Znaczenie
OWASP	Open Worldwide Application Security Project — społeczność i fundacja tworząca otwarte materiały dotyczące bezpieczeństwa aplikacji.
OWASP Top 10	lista najważniejszych klas ryzyka bezpieczeństwa aplikacji webowych.
ASVS	Application Security Verification Standard — standard wymagań służący do weryfikacji bezpieczeństwa aplikacji.
WSTG	Web Security Testing Guide — przewodnik po testowaniu bezpieczeństwa aplikacji webowych.
NIST	National Institute of Standards and Technology — amerykański instytut opracowujący standardy i wytyczne techniczne.
SSDF	Secure Software Development Framework — ramy bezpiecznego procesu wytwarzania oprogramowania.
W3C	World Wide Web Consortium — organizacja tworząca standardy technologii internetowych.
MFA	uwierzytelnianie wieloskładnikowe; wykorzystanie co najmniej dwóch niezależnych czynników.
TOTP	jednorazowy kod czasowy generowany np. przez aplikację uwierzytelniającą.
Passkey / klucz dostępu	poświadczenie kryptograficzne pozwalające uwierzytelniać użytkownika bez klasycznego hasła.
WebAuthn	standard przeglądarkowy do uwierzytelniania z użyciem kluczy kryptograficznych.
RBAC	Role-Based Access Control — kontrola dostępu oparta na rolach.
ABAC	Attribute-Based Access Control — kontrola dostępu oparta na atrybutach użytkownika, zasobu lub kontekstu.
BOLA	Broken Object Level Authorization — błąd autoryzacji pozwalający uzyskać dostęp do cudzego obiektu.
IDOR	Insecure Direct Object Reference — niebezpieczne bezpośrednie odwołanie do obiektu przez identyfikator bez właściwej kontroli dostępu.
API	Application Programming Interface — interfejs programistyczny udostępniający funkcje i dane aplikacji.
REST / GraphQL	dwa popularne sposoby projektowania interfejsów API.
SBOM	Software Bill of Materials — wykaz składników i zależności użytych w oprogramowaniu.

SCA	Software Composition Analysis — analiza bezpieczeństwa bibliotek i zależności.
CVE	publiczny identyfikator znanej podatności.
CWE	katalog typów słabości i błędów w oprogramowaniu.
CVSS	system punktowej oceny technicznej istotności podatności.
CI/CD	ciągła integracja i dostarczanie/wdrażanie — automatyzacja budowania, testowania i publikowania oprogramowania.
SAST	statyczna analiza bezpieczeństwa kodu — test bez uruchamiania aplikacji.
DAST	dynamiczne testowanie bezpieczeństwa — test działającej aplikacji.
DevSecOps	włączanie bezpieczeństwa w cały proces tworzenia i utrzymania oprogramowania.
MIME	informacja opisująca typ zawartości pliku lub danych przesyłanych przez HTTP.
Path traversal	manipulacja ścieżką w celu odczytu lub zapisu poza dozwolonym katalogiem.
XSS	Cross-Site Scripting — wstrzyknięcie kodu wykonywanego w przeglądarce użytkownika.
CSRF	Cross-Site Request Forgery — wymuszenie na zalogowanej przeglądarce wykonania niezamierzonego żądania.
CSP	Content Security Policy — polityka przeglądarki ograniczająca źródła i sposoby ładowania treści/skryptów.
SSRF	Server-Side Request Forgery — wymuszenie na serwerze wykonania żądania do wskazanego przez atakującego celu.
Allowlist	lista jawnie dozwolonych wartości/celów; wszystko poza listą jest odrzucane.
Egress	ruch wychodzący z systemu lub sieci.
Hardening / utwardzanie	zmniejszanie powierzchni ataku poprzez bezpieczniejszą konfigurację i wyłączenie zbędnych funkcji.
Retest / test ponowny	powtórzenie testu po poprawce w celu potwierdzenia, że problem został usunięty.
Evidence / materiał dowodowy	zrzuty, żądania HTTP, logi, wyniki testów, fragmenty kodu lub inne dane potwierdzające wniosek.
Endpoint	konkretny punkt/funkcja interfejsu API dostępna pod określonym adresem lub operacją.
Backend	część serwerowa aplikacji realizująca logikę biznesową i dostęp do danych.

Origin	źródło strony w przeglądarce określone przez schemat, domenę i port.
Replay	ponowne odtworzenie wcześniej przechwyconego komunikatu lub operacji.
Fallback	metoda zapasowa, np. alternatywny sposób odzyskania dostępu.
Tenant	odrębny klient/organizacja obsługiwana w ramach jednej wspólnej aplikacji.

1. Raport 8–12 stron	cel i zakres, środowisko, metodyka, wyniki, dowody, mapowanie do standardów, ocena ryzyka, remediacja, test ponowny, wnioski.
2. Repozytorium / materiały	kod, konfiguracje, skrypty testowe, instrukcja uruchomienia, diagramy oraz pozostałe dowody.
3. Prezentacja 10–12 minut	problem → metoda → dowód BEFORE → poprawka → wynik AFTER → najważniejsze wnioski.
4. Demonstracja	krótki, powtarzalny scenariusz techniczny pokazujący najważniejszy element projektu.
5. Karta projektu	oddzielny dokument z zespołem, zakresem, środowiskiem, hipotezą, scenariuszami i odniesieniami do standardów.

Materiały referencyjne

OWASP Top 10:2025: https://top10.owasp.org/2025/

OWASP ASVS: https://owasp.org/projects/asvs

OWASP WSTG: https://wstg.owasp.org/

OWASP API Security: https://api-security.owasp.org/

NIST SP 800-218 — SSDF: https://csrc.nist.gov/pubs/sp/800/218/final

W3C WebAuthn Level 3: https://www.w3.org/TR/webauthn-3/

ZASADA KOŃCOWA

Projekt zostaje zaakceptowany wtedy, gdy zespół potrafi odtworzyć środowisko, pokazać problem lub kontrolę, uzasadnić ryzyko, wdrożyć zabezpieczenie oraz potwierdzić jego skuteczność w teście ponownym.